CS5013 Project Design Document

GUI Framework for Multi-Tool Panoramic Image Segmentation

Rathan Balaji P R (CS26E009)
Department of Computer Science and Engineering, IIT Madras

September 10, 2026

1 Design objectives and scope

The submitted proposal defines a Java desktop GUI for integrating multiple heterogeneous image-segmentation tools on 360° panoramic images. The design below turns that scope into explicit modules, interfaces, data flow, and tests. The architecture deliberately keeps model execution separate from the Java GUI so that tools implemented in other runtimes can be invoked through a controlled process or HTTP boundary.

The principal design goals are: (i) one user workflow for selecting a panoramic input and segmentation tool(s); (ii) a common adapter contract for model-specific preprocessing, execution, and post-processing; (iii) a canonical segmentation-result representation for visualization and comparison; (iv) panorama-aware display of masks and overlays; and (v) quantitative comparison only when ground truth and compatible label mappings are available.

2 System architecture

The system is organized as a layered architecture. The GUI and application-control layer do not contain model-specific code. Each external segmentation tool is isolated behind an adapter implementing the common `SegmentationTool` abstraction. This allows the core workflow and comparison modules to remain unchanged when another tool is added.

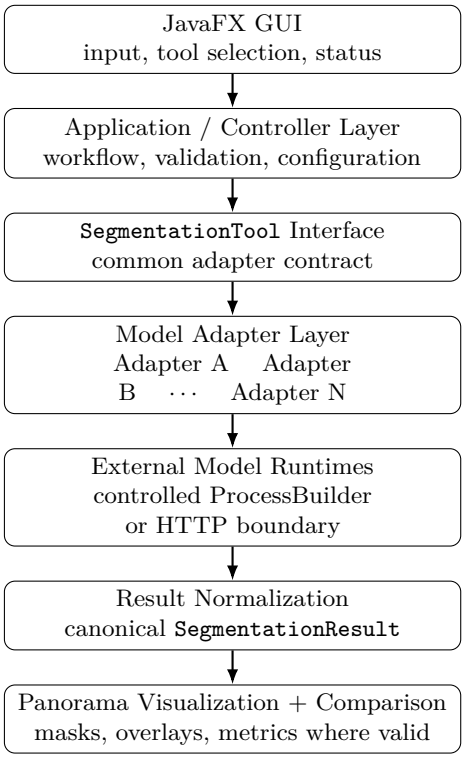


Figure 1: Proposed layered architecture and principal data flow.

2.1 Main interfaces

- **`SegmentationTool`:** common contract for a tool adapter. It exposes tool metadata/configuration and a controlled segmentation operation whose output is converted to the canonical result representation.
- **`SegmentationRequest`:** carries the selected panoramic input and tool-specific configuration needed by an adapter.
- **`SegmentationResult`:** canonical representation consumed by the GUI and comparison layer. It preserves relevant semantic class or instance information, mask dimensions, and result metadata.
- **`SegmentationAdapter`:** implementation boundary for model-specific preprocessing, external execution, post-processing, and error translation. Individual adapters may communicate with a process or HTTP inference service.

2.2 Execution flow

A user selects a supported equirectangular panorama and one or more registered tools. The controller validates the input and configuration, creates a request, and dispatches it to the selected adapter(s). Each adapter performs the model-specific preprocessing and invokes its external runtime. Returned outputs are post-processed into `SegmentationResult`. The visualization layer displays the original panorama, masks, and overlays; the comparison layer aligns compatible results and computes metrics when valid ground truth is available.

3 Module split and responsibilities

The following module split is intentionally aligned with the proposed Java/JavaFX stack and the adapter architecture in the submitted proposal.

Module	Responsibility	Key interface / dependency
GUI / Presentation	Load panorama, select tools, show progress/errors, display masks, overlays, and comparison results.	JavaFX controllers, image/canvas components.
Application / Controller	Own the user workflow, validate requests, coordinate adapters, and manage execution state.	`SegmentationRequest`, `SegmentationTool`.
Tool Registry / Configuration	Discover/register adapters and load model/tool configuration without changing the core GUI.	Adapter metadata and configuration files.
Segmentation Interface	Define the common contract used by all integrated tools.	`SegmentationTool`.
Model Adapters	Perform model-specific preprocessing, execution, post-processing, and error translation.	`SegmentationTool`; ProcessBuilder or HTTP.
Result Normalization	Convert heterogeneous outputs into a canonical representation while preserving relevant class/instance information.	`SegmentationResult`.
Panorama Visualization	Display the equirectangular image, masks, overlays, and selected comparison views; account for panorama-specific visual issues.	JavaFX rendering components.
Comparison / Metrics	Present outputs in a consistent format and compute mIoU, pixel accuracy, Dice/F1, and per-class IoU when valid ground truth and label mappings exist.	`SegmentationResult`; ground-truth mask.
Validation / Error Handling	Validate files/configuration, detect failed or empty model results and dimension mismatches, and surface clear errors.	Shared validation/error types.

3.1 Canonical result representation

The canonical result is the key interoperability boundary. At minimum it should contain the output image dimensions, segmentation type, a mask or mask collection, class/instance identifiers where available, and metadata describing the producing tool. The representation must not silently discard information that cannot be mapped; unsupported information should be explicitly marked. This supports visual comparison while preventing incompatible outputs from being treated as directly equivalent.

3.2 External execution boundary

Model runtimes may have Python dependencies, GPU requirements, or incompatible libraries. They therefore remain outside the Java GUI process. `ProcessBuilder` will be used for controlled local process execution where appropriate; HTTP communication will be used where an inference service is exposed. The adapter hides this distinction from the rest of the application.

3.3 Panorama-specific handling

The initial data representation is an equirectangular 360° image. Visualization must make the horizontal wrap-around explicit because the two horizontal boundaries represent adjacent directions in the panorama. Where direct comparison is visually misleading because of projection distortion, the design permits selected regions or transformed views in addition to the complete equirectangular view.

4 Test plan

The test plan follows the course requirement of at least one concrete test per module. Tests are grouped into unit, integration, and end-to-end levels.

Module	Test case	Expected result
GUI / Presentation	Load a valid equirectangular panorama and select an integrated tool.	Image appears; controls and execution state are usable.
Application / Controller	Submit a valid request and verify dispatch to the selected adapter.	Correct adapter receives the request and execution state is updated.
Tool Registry / Configuration	Load valid configuration and reject missing/invalid configuration.	Valid tools register; invalid configuration produces a clear error.
Segmentation Interface	Run an adapter through the common contract using a fixed test image.	Adapter satisfies the contract and returns a result or a controlled error.
Model Adapter	Execute each integrated adapter on a fixed panorama.	Model-specific runtime is invoked and output is converted to the canonical result.
Result Normalization	Feed representative heterogeneous output formats to the normalizer.	Canonical result is produced with dimensions and relevant labels preserved.
Panorama Visualization	Display a mask overlay, including a case crossing the horizontal boundary.	Overlay is rendered consistently without silently losing the wrap-around region.
Comparison / Metrics	Compare selected outputs against a fixed ground-truth mask.	Metrics match independently computed reference values for compatible labels.
Validation / Error Handling	Supply invalid path, unsupported format, failed execution, empty output, and mismatched dimensions.	Clear error state is shown; invalid results are not presented as valid segmentation.

4.1 Integration and end-to-end verification

Integration tests will execute each model adapter on fixed test images and verify that a standardized result is produced. End-to-end tests will cover the complete workflow from panoramic image selection through inference, visualization, and comparison. Quantitative metrics will be checked against independently computed reference values for selected cases. Ground-truth evaluation will be performed only for samples with valid masks and compatible label mappings.

5 Revised milestone plan

The proposal established the major dates. The design-stage plan below refines those milestones into implementation checkpoints while preserving the same scope. If course feedback requires scope reduction, the initial model set can be reduced without changing the common adapter architecture.

Period	Planned work	Deliverable / evidence
By 11 Sep 2026	Finalize architecture, module boundaries, common interfaces, canonical result representation, test plan, and revised milestones.	Design document and runnable project skeleton.
12–20 Sep	Implement JavaFX application shell, configuration/tool registry, request/result data structures, and the `SegmentationTool` interface.	GUI skeleton and tested core interfaces.
21 Sep–2 Oct	Implement the first end-to-end adapter and external execution boundary; add result normalization and basic error handling.	One complete panorama-to-segmentation workflow.
3–8 Oct	Integrate additional selected tools, improve panorama visualization, and implement the initial comparison view.	Multi-tool workflow ready for demonstration.
9 Oct 2026	Mid-demo: demonstrate the working core workflow on real panoramic images with multiple integrated tools.	Live end-to-end demonstration and Git history evidence.
10 Oct–23 Oct	Harden adapters, improve canonical result handling, asynchronous execution, progress/error states, and caching where required.	More robust multi-tool execution.
24 Oct–5 Nov	Complete comparison/metrics, automated tests, documentation, and instructions for adding another adapter; perform stakeholder dry-run.	Final runnable package and verification results.
6 Nov 2026	Final submission and viva.	Complete GUI, documentation, tests, and stakeholder-executed demonstration/video.

6 Risks and plan B

1. **Runtime/dependency incompatibility:** A selected model may require a GPU, Python environment, or library versions that cannot be embedded in Java. **Plan B:** keep execution external behind ProcessBuilder/HTTP; if necessary, reduce the initial model set while preserving the adapter architecture.
2. **Output incompatibility:** Tools may return different mask representations, class identifiers, or dimensions. **Plan B:** enforce the canonical result schema and model-specific post-processing; explicitly mark unsupported information.
3. **Panorama distortion:** Direct visual comparison can be misleading near the horizontal wrap-around boundary. **Plan B:** provide panorama-aware visualization and selected-region/transformed-view inspection where needed.
4. **Inference latency:** Some models may be too slow for interactive use. **Plan B:** asynchronous execution, progress/error states, caching of completed results, and a small representative model set for the live demo.
5. **Ground-truth availability/compatibility:** Ground truth may be missing or use incompatible labels. **Plan B:** keep visual comparison independent and compute quantitative metrics only for valid, compatible samples.

7 Design decisions and completion criteria

The central design decision is to treat the segmentation model as a replaceable service behind a stable Java interface. This prevents model-specific dependencies from leaking into the GUI and makes the framework extensible. The initial implementation will prioritize a small set of representative tools rather than attempting to support every available model.

The design will be considered complete for the mid-demo when a user can select a real 360° panoramic image, run multiple integrated segmentation tools through the GUI, and view their standardized outputs in a common comparison workflow. The final implementation additionally requires robust error handling, automated tests, quantitative comparison where ground truth is valid, documentation, and a clear procedure for integrating another tool.